

ShopEZ – E-Commerce Fashion Website

by Pamu Harsha Vardhan B-3

Mentor

Narsimha

Introduction

The E-Commerce Fashion Website is a full-stack web application developed to provide users with an online platform for browsing, searching, purchasing, and managing fashion products.

The application follows a Microservices Architecture where each service is independently developed and managed. The frontend is developed using Angular, while the backend uses ASP.NET Core Web API with SQL Server databases.

The system includes features such as:

- User Authentication
- Product Management
- Category Filtering
- Shopping Cart
- Order Management
- Payment Handling
- Admin Dashboard
- API Gateway Integration
- Docker Containerization

Objectives:

Main Objectives

- Develop a scalable e-commerce platform.
- Implement microservices architecture.
- Provide secure authentication and authorization.
- Enable online product browsing and purchasing.
- Improve maintainability and scalability.

Secondary Objectives

- Learn Docker containerization.
- Implement API Gateway.
- Apply frontend-backend integration.
- Understand distributed application architecture.

Technologies Used:

Technology	Purpose
Angular	Frontend Development
ASP.NET Core Web API	Backend APIs
SQL Server	DataBase
Entity Framework Core	ORM
Docker	Containerization
Ocelot API Gateway	API Gateway
JWT Authentication	Security
Bootstrap	UI Design
Git & GitHub	Version Control

System Architecture

Architecture Overview

The application follows a Microservices Architecture.

Main Components

1. Frontend (Angular)
2. API Gateway (Ocelot)
3. User Service
4. Product Service
5. Order Service
6. SQL Server Databases
7. Docker Containers

Architecture Flow:

User



Angular Frontend



API Gateway



| User Service |

| Product Service |

| Order Service |



SQL Server Databases

Frontend Module Description

Angular Frontend

The frontend provides a responsive user interface.

Main Components:

- Home Component
- Product Component
- Product Details Component
- Cart Component
- Checkout Component
- Login/Register Component
- Admin Dashboard
- Header Component
- Footer Component

Features:

- Product Search
- Category Filtering
- JWT Token Handling
- Responsive Design

Backend Microservices

User Service

Handles:

- User Registration
- Login Authentication
- JWT Token Generation
- Role Management

APIs

Method	Endpoint	Description
GET	/api/User	Retrieve All user
GET	/api/User{id}	Retrieve User By id
POST	/api/User/register	User Register
POST	/api/User/login	User Login
PUT	/api/User/{id}	Update User By id
DELETE	/api/User/{id}	Delete User By id

Product Service

Handles:

- Product CRUD Operations
- Product Images
- Categories

APIs

Method	Endpoint	Description
GET	/api/Product	Get All Products
GET	/api/Product/{id}	Get Products By Id
POST	/api/Product	Add Product
PUT	/api/Product/{id}	Update Product By Id
DELETE	/api/Product/{id}	Delete Product By Id

Order Service:

- Adding Order
- Retrieving All Orders
- Retrieving Order By Id

Apis:

Method	EndPoint	Description
GET	/api/Order	Get all Orders
GET	/api/Order/{id}	Get Order by id
POST	/api/Order	Add Order

API Gateway:

Ocelot API Gateway

The API Gateway acts as a single entry point for all microservices.

Responsibilities

- Request Routing
- Authentication Validation
- Load Distribution
- Simplified Client Communication

Example Routing:

```
{  
  "UpstreamPathTemplate": "/gateway/products",  
  "DownstreamPathTemplate": "/api/products",  
  "DownstreamHostAndPorts": [  
    {  
      "Host": "productservice",  
      "Port": 8080  
    }  
  ]  
}
```

Authentication & Security:

Security Features

- JWT Authentication
- Password Hashing
- Role-Based Authorization
- Secure API Access

Authentication Flow

User Login



JWT Token Generated



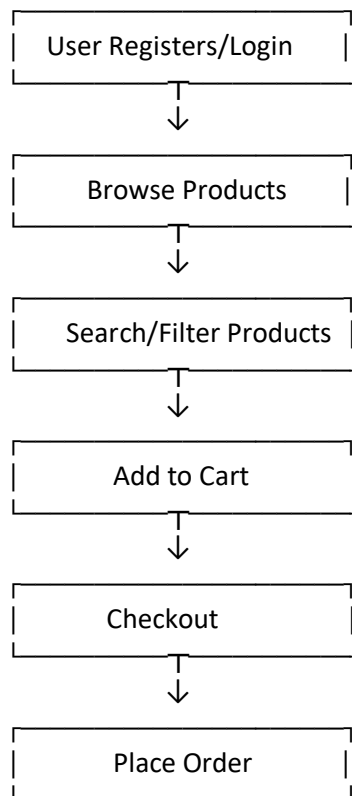
Frontend Stores Token



Token Sent in API Requests

User Flow:

Customer Flow



Admin Flow

1. Login as Admin
2. Add/Edit/Delete Products
3. Manage Orders

SOFTWARE TESTING

Frontend Testing

Unit Testing using Karma & Jasmine

I performed unit testing for Angular components using **Karma** and **Jasmine** frameworks.

The testing was done to verify whether the frontend components were working correctly without errors.

Components Tested

- Header Component
- Products Component
- Checkout/Cart Component

Testing Performed

1. Header Component Testing

Test cases verified:

- Component creation
- Navigation to Home page
- Navigation to Products page
- Navigation to Cart page

Result:

- All 4 test cases passed successfully.

2. Products Component Testing

Test cases verified:

- Products array loading
- Category selection
- Gender filter selection
- Clear filters functionality
- Component creation

Result:

- All 5 test cases passed successfully.

3. Checkout/Cart Component Testing

Test cases verified:

- Cart items array presence
- User object validation
- Total amount calculation
- Order placement functionality
- Component creation

Result:

- All 5 test cases passed successfully.

Tools Used

- Angular Testing Framework
- Karma Test Runner
- Jasmine Testing Framework
- Chrome Browser for execution

Karma v 6.4.4 - connected; test: complete;

Chrome 148.0.0.0 (Windows 10) is idle

 Jasmine 4.6.1

5 specs, 0 failures, randomized with seed 29972

finished

Spec List | [Performance](#)

Checkout

- should have cart items array (120ms)
- should create (11ms)
- should have user object (7ms)
- should have total amount (7ms)
- should place order (15ms)

Karma v 6.4.4 - connected; test: complete;

Chrome 148.0.0.0 (Windows 10) is idle

 Jasmine 4.6.1

4 specs, 0 failures, randomized with seed 56691

Spec List | [Performance](#)

Header

- should create (195ms)
- should navigate to home page (16ms)
- should navigate to products page (13ms)
- should navigate to cart page (41ms)

Karma v 6.4.4 - connected; test: complete;

Chrome 148.0.0.0 (Windows 10) is idle

 Jasmine 4.6.1

5 specs, 0 failures, randomized with seed 50122

Spec List | [Performance](#)

Products

- should have products array (252ms)
- should select gender (11ms)
- should clear filters (11ms)
- should select category (10ms)
- should create (8ms)

Backend Testing

Backend Test Cases Implemented

User Service Testing

- Login with valid credentials

Product Service Testing

- Create Product
- Get Products

Order Service Testing

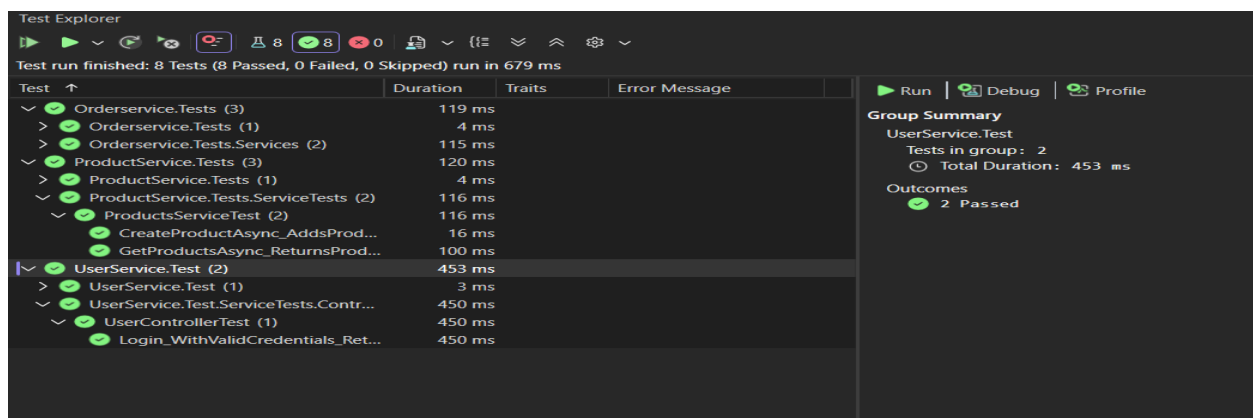
- Create order
- Calculate total amount

Mocking Implementation

Used **Moq** framework to mock:

- Repository layer
- JWT Service
- Database operations

This helped isolate business logic during testing without using the actual database.

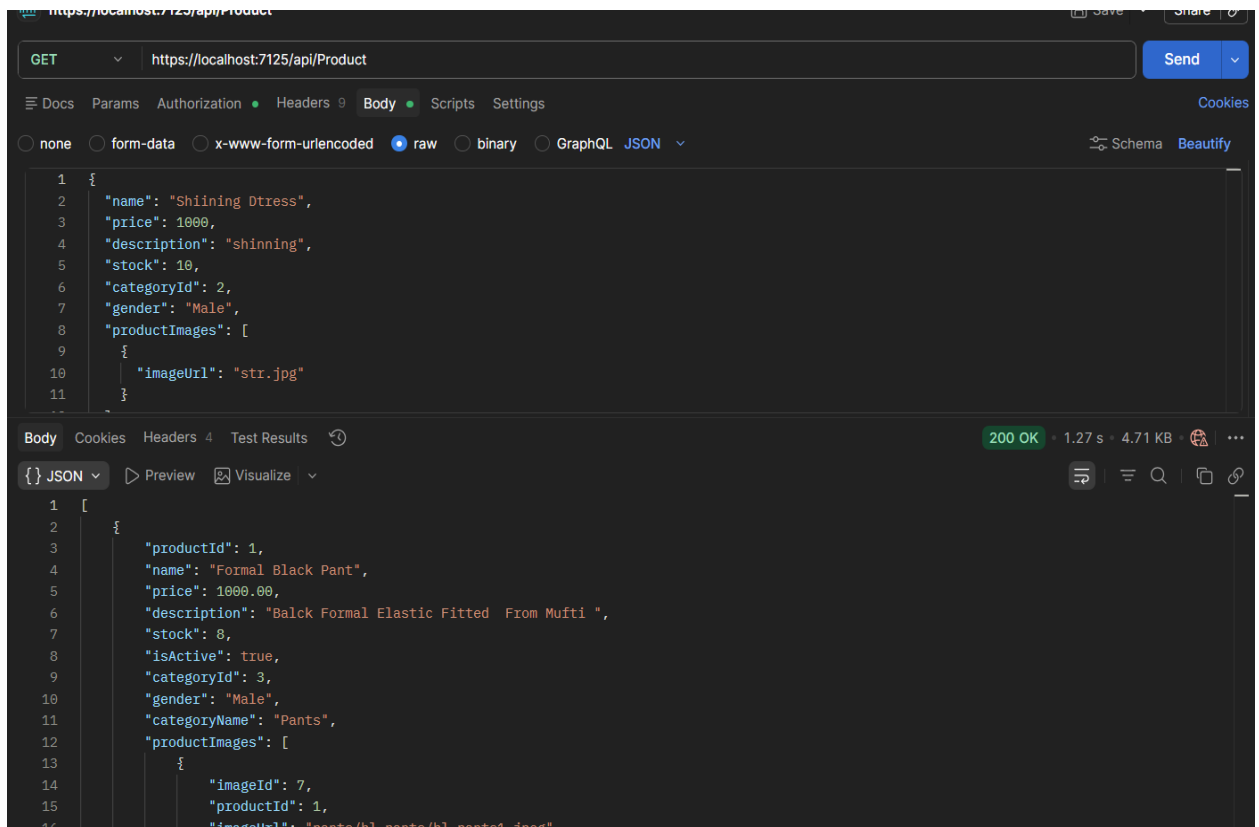


API Testing using Postman

API testing was performed using Postman to validate User, Product, and Order Service APIs. JWT-based authentication and authorization were tested using Bearer Tokens. Operations such as user registration, login, product creation, product retrieval, product deletion, and order creation were successfully validated.

APIs Tested

- User Service API
- Product Service API
- Order Service API
- Authentication APIs



POST https://localhost:7125/api/Product Send

Docs Params Authorization Headers Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "name": "Shining Dtrass",
3   "price": 1000,
4   "description": "shinning",
5   "stock": 10,
6   "categoryId": 2,
7   "gender": "Male",
8   "productImages": [
9     {
10      "imageUrl": "str.jpg"
11    }
12  ]
13 }
```

Body Cookies Headers 5 Test Results 201 Created · 1.51 s · 501 B · ⌵ ⋮

{} JSON Preview Visualize ⌵

POST https://localhost:72 GET https://localhost:7125/t No environment ⌵ ⋮

https://localhost:7125/api/Product Save Share ⌵ ⋮

GET https://localhost:7125/api/Product Send

Docs Params Authorization Headers Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "name": "Shining Dtrass",
3   "price": 1000,
4   "description": "shinning",
5   "stock": 10,
6   "categoryId": 2,
7   "gender": "Male",
8   "productImages": [
9     {
10      "imageUrl": "str.jpg"
11    }
12  ]
13 }
```

Body Cookies Headers 4 Test Results 200 OK · 99 ms · 5.02 KB · ⌵ ⋮

{} JSON Preview Visualize ⌵

```
346 {
347   "productId": 11,
348   "name": "Shining Dtrass",
349   "price": 1000.00,
350   "description": "shinning",
351   "stock": 10,
352   "isActive": true,
353   "categoryId": 2,
354   "gender": "Male",
355   "categoryName": "Shirts",
356   "productImages": [
357     {
358       "imageId": 44,
359       "productId": 11,
360       "imageUrl": "str.jpg"
361     }
362   ]
363 }
```


POST https://localhost:7236/api/User/register

POST https://localhost:7236/api/User/register

Body

```
1 {
2   "email": "bro@gmail.com",
3   "fullName": "Brocode",
4   "mobileNo": "9632587410",
5   "password": "Bro@123"
6 }
```

200 OK • 2.06 s • 176 B

Body

```
1 {
2   "message": "User registered successfully"
3 }
```

GET https://localhost:7151/api/Order

GET https://localhost:7151/api/Order

Body

```
1 {
2   "userId": 1,
3   "fullName": "Admin",
4   "email": "admin@gmail.com",
5   "address": "sircilla",
6   "totalAmount": 50000,
7   "items": [
8     {
9       "productId": 1,
10      "productName": "xxx",
11      "selectedSize": "M",
12      "selectedColor": "Black",
13      "selectedSize": "M",
14      "selectedColor": "Black"
15     }
16   ]
17 }
```

200 OK • 3.71 s • 1.89 KB

Important Notification Request

Allow important notifications from Snipping Tool ?
You will get important notifications from Snipping Tool when do not disturb is on. To change this later, select this app in notification settings.

Docker Containerization

Purpose of Docker

Docker is used to containerize the frontend, backend services, and database.

Docker Components

- Angular Container
- User Service Container
- Product Service Container
- Order Service Container
- SQL Server Container
- API Gateway Container

Docker Workflow

Dockerfile → Docker Image → Docker Container

Docker Compose

Docker Compose is used to manage all containers together.

Example Services

- frontend
- gateway
- userservice
- productservice
- orderservice
- sqlserver

Project SnapShots

ShopEZ

Search products...

Q

Home

Products

Bag

Login

LoginSign Up

Login

Email

Password

Login

ShopEZ

Best online store for fashion and lifestyle products.

Quick Links

Home

Products

Cart

Contact

support@shopez.com

+91 98765 43210

Download App

GET IT ON Google Play

Download on the App Store

Follow Us

f

t

y

i

ShopEZ

Search products...

Q

Home

Products

Bag

Login

LoginSign Up

Sign Up

Full Name

Email

Password

Mobile

Register

ShopEZ

Best online store for fashion and lifestyle

Quick Links

Home

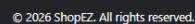
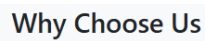
Contact

support@shopez.com

Download App

GET IT ON Google Play

Download on the App Store



ShopEZ

Search products...

HomeProductsBagHarsha

Categories

All

Pants

Shirts

Footwear

T-Shirts

Hoodies

Gender

Male

Female

Our Products



Formal Black Pant

₹1000

In Stock (7)

View



White Shirt H&M

₹2000

In Stock (3)

Only 3 left!

View



Sky Blue Shirt

₹2000

In Stock (4)

View



White Shoes PUMA

₹3000

In Stock (5)

View

← Back to Products









Product Details



Sky Blue Shirt

₹2000

Linen Sky Blue Oversized Men Shirt From levis

Stock: 3

Male

Select Size

L

XL

Add to Cart

[Home](#)
[Products](#)
[Bag](#)
[Harsha](#)

Your Cart

[← Back to Products](#)

Sky Blue Shirt

₹2000

Size: L

-

1

+

Remove

Total: ₹2000

Proceed to Checkout

ShopEZ

Best online store for fashion and lifestyle products.

Quick Links

Home

Products

Cart

Contact

support@shopez.com

+91 98765 43210

Download App

GET IT ON Google Play

Download on the App Store

Follow Us

100% ORIGINAL guarantee for all products

Return within 7 days of receiving your order

[Home](#)
[Products](#)
[Bag](#)
[Harsha](#)

Delivery Details

Order Summary

Sky Blue Shirt

₹ 2000 x 1

Total: ₹ 2000

Place Order

ShopEZ

Best online store for fashion and lifestyle products.

Quick Links

Home

Products

Cart

Contact

support@shopez.com

+91 98765 43210

Download App

GET IT ON Google Play

Download on the App Store

Follow Us

100% ORIGINAL guarantee for all products

Return within 7 days of receiving your order

© 2026 ShopEZ. All rights reserved.

Note:videoLink

(https://drive.google.com/file/d/1H2aFPvWd7X9jtyhcUNu8D9pHX_4FBtrP/view?usp=sharing)

Challenges Faced

- Managing communication between microservices.
- Handling Docker networking.
- Configuring API Gateway routing.
- JWT Authentication integration.
- Managing multiple databases.

Future Enhancements

- Online Payment Integration
- Email Notifications
- Product Recommendation System
- AI-Based Search
- Cloud Deployment on Azure/AWS
- Kubernetes Deployment

Conclusion

The E-Commerce Fashion Website successfully demonstrates the implementation of a modern full-stack microservices-based application using Angular and ASP.NET Core.

The project provides scalability, flexibility, security, and maintainability while offering a responsive shopping experience for users.

This project helped in understanding:

- Microservices Architecture
- API Gateway
- Docker Containerization
- Frontend-Backend Integration
- Authentication & Authorization
- Database Management